Presentation Project Guideline

1. Introduction

In this final project, you will enhance your distributed chat system by adding a Graphical User Interface (GUI) and one or more feature extensions. These extensions will turn your simple terminal-based chat system into a more complete application similar to the chat apps we use in daily life.

All project features must be implemented on top of your own chat system, which includes a server and multiple clients connected through sockets.

Teams:

- Two-person team → GUI + one selective topic
- Three-person team → GUI + two selective topics
- Note: you should find your team by April 22th and register your team in this table: ICDS Spring 2026 Final Project Teams ; cross-section teams are **allowed**.

AI-assisted Development Requirements:
Students are required to use pi-mono during the development of their final project. Possible usages include:

- generating code
- debugging modules
- testing functions
- summarizing documentation

Each team must demonstrate at least one meaningful usage of pi-mono in their presentation video.

(To install pi-mono, please refer to pi-mono Installation & Quick Start Guide)

What You Need to Submit
In Week 14, by May 10th, you will submit your final deliverables including the source code, a 10–15 minute video, and associated documentation.

1. Project Code
   - Server, client, GUI, and other modules
2. Presentation Video (10–15 min)
   - Must include introduction, demo, discussion, and reflection.
   - Must how pi-mono was used during development.
3. Presentation slides
   - Include GUI and feature screenshots.
   - Member Contribution (optional)

2. Compulsory Topic: Build a Graphic User Interface (GUI)

Currently, the chat system is run from the terminal command line. All input and output are only found from the plain terminal window. You should add a user interface to enhance the chatting experience.

The goal: To construct an interface using Python packages like Tkinter so that clients do not need to type or read from the terminal window when connecting to the chat server. All information being exchanged will be displayed on the GUI. (**Note:** we will provide you with a sample GUI chat system. You can start with it.)

1. Minimum Requirement of the GUI.
   - A window for displaying messages
   - A text input area
   - A Send button
   - Real-time message updates
   - Clear message formatting

2 Extra Features (You can implement one or more of the optional enhancements)
   - login, emoji, file transfer, buttons for time/who/poem/search, voice/video chat.

3 Integrate Your Selective Topic
   - Example: a button to start the game or chatbot interaction.

Note for Students:

1. Displaying both sent and received messages is required for a complete GUI experience.

👉 The GUI chat interface must display both the messages sent by the user and those received from others, similar to real-world chat applications.

It is not acceptable to only display incoming messages or only display messages sent by oneself.

This is considered a basic requirement for a complete GUI design.

2. The provided GUI sample is only a minimal template and intentionally contains a small bug related to message resetting.

👉 The sample GUI code provided in the project materials is only a basic template and not a final or complete version.

It intentionally contains a small bug: after a message is displayed in the chat window, the variable self.system_msg is not reset to an empty string, which may cause duplicated or repeated display of messages.

This bug is left intentionally for students to discover and fix by examining the message handling logic.

3. Extra features are graded based on successful implementation and proper functionality, not on subjective complexity or visual polish.

👉 For extra features (such as emoji, login, file transfer, search buttons, voice/video chat, etc.),

as long as the feature is correctly implemented and functions properly, full credit will be awarded.

There is no additional score for visual appearance, complexity, or advanced styling.

Students are encouraged to submit a version they personally consider functional, stable, and satisfactory, rather than focusing on visual decoration or over-engineering.

3. Selective Topics

3.1 Chatbot

Large Language Models (LLMs) are among the most advanced technologies available today, and many pretrained models can be readily used to build intelligent applications. For this project, you may choose any publicly accessible LLM to develop a chatbot for your chat system. (**Note:** To help you get started, we will provide a **`ChatBotClient`** class as a basic template.)

There are two possible ways to integrate a chatbot into your system:

1. Running the chatbot client **on the chat server**, or
2. Running the chatbot client **on the client side**.

Both approaches are valid, and each comes with its own design implications. Consider the advantages of each model and choose the one that best fits your project's architecture and goals.

This topic includes three main components:

1. **Basic Chatbot Functionality**
   Create a chatbot that can carry out conversations with a single client—similar to how ChatGPT interacts with users. When the client sends a message, the chatbot should generate an appropriate response. The conversation may look like this:

```
You: What is the capital of France?
Chatbot: The capital of France is Paris.
```

2. **Context Awareness and Personality**
   The chatbot should maintain conversation context, meaning it should remember previous messages and use that information when generating new responses. Additionally, users should be able to assign a "personality" to the chatbot, influencing its tone and behavior.

3. **Bonus: Participation in Group Chats**
   As an advanced feature, the chatbot may be invited into a multi-client group chat. In this scenario, it should recognize when to respond and integrate naturally into the group conversation. The conversation may look like this:

   ```
   Alice: @chatbot what do you think about our project?
   Chatbot: I think your project is progressing well!
   ```

   The chatbot can participate naturally in a multi-client group conversation, responding appropriately when mentioned or addressed.

Hints:

- You may use **Ollama** to run an LLM locally or call online LLM APIs (e.g., OpenAI). **Note:** We will provide you an online LLM.
- Messages sent to and received from the LLM should follow a consistent format, similar to your chat system's message protocol.
- The following resources can help you understand how to communicate with Ollama:
  - Introduction - Ollama
  - Ollama Python library

Notes of Chatbot:

Using the Provided AI Code Files

We have provided two example files for implementing AI ChatBot functionality:

1. chat_bot_client.py

This file contains two classes for AI chatbot functionality:

Class	Description
ChatBotClient	Uses Ollama local models (e.g., phi3, llama3, qwen, etc.)
ChatBotClientOpenAI	Uses OpenAI-style APIs, such as ChatGPT API or any API with OpenAI-compatible format

You only need **one** of these two methods — pick whichever is suitable for your environment.

2. ai_client.py

This file uses our **locally deployed Qwen model API**, which already follows the OpenAI-compatible format.

You can directly call the API in this file without making structural changes.

The example files only demonstrate the basic API calling functions. If you want to build more advanced features or custom chatbot capabilities, you are encouraged to further explore, enhance, and extend these implementations based on your project goals.

📌 Importing Classes or Functions from Another .py File

If you download chat_bot_client.py and want to use the ChatBotClient class inside **another Python file**, you can import it using:

```
from chat_bot_client import ChatBotClient
```

After that, you can create an instance:

```
bot = ChatBotClient()
reply = bot.chat("Hello, who are you?")
```

Important Notes When Importing Between Files

- The .py files must be in the **same directory (folder)** or within the same Python package. File names **must not conflict** with standard library module names (e.g., do **not** name your file sys.py, json.py, numpy.py, etc.).

Using separate files and proper imports can help make your Final Project **more modular, organized, and easier to maintain**.

3.2 Online Gaming

Online gaming is one of the fastest-growing digital industries today. It is a broad, interdisciplinary field that brings together artists and creative professionals (for asset creation and storytelling), business teams (marketing, distribution, logistics), and, of course, skilled computer scientists (responsible for game engines, networking infrastructure, implementation, and quality assurance).

This project topic is designed to give you a small but meaningful glimpse into the technical

challenges behind building even a simple online game. Your game will use the existing chat system as its communication backbone.

In the current chat system, clients send messages to the server, the server forwards these messages to one or more clients, and clients reply through the same channel. The server essentially acts as the "man in the middle," maintaining all communication and ensuring messages reach the correct destinations.

Your task is to adapt this communication model to support a simple game platform. You should choose a simple and small game you find online—for example, a Pygame demo or a **Tkinter-based** game (recommended) from GitHub—so long as it can be integrated with the chat system. There are two options:

- **Basic: Single player game with ranked scores**
  A game that can be played individually, where each player's final score is sent to the chat server. The server will maintain a scoreboard, sort the scores, and broadcast updated rankings to all connected clients. For example:
  - You can integrate the game like Snake. Each player runs the game independently, and once the game ends, the score is reported to the chat server. The server then updates a global leaderboard—effectively creating a shared competitive environment even though gameplay is single-player.
- **Bonus**: **Interactive Multiplayer Gaming**
  A game that supports interactive play between two or more clients—for example:
  - A **Tic-Tac-Toe** or chess-like two-player game running in its own graphical window (note: it should *not* be text-based).
  - A simple racing or action game where multiple players participate simultaneously in the same game environment.

  During gameplay, two clients will send their moves to the server. The server should transmit the moves to the other players. Interestingly, this pattern of interaction is conceptually similar to how many real-world online games operate: the server enforces game rules and synchronizes all players, while the clients focus on rendering the game and sending player actions.

3.3 Bonus topics

There are three extra bonus topics. If you complete any or all of them, you can obtain more bonus points. (See the grading policy for details.)

- **Topic 1: AI Picture Generation**
  Allow users to generate AI images directly in the chat by using a special input format. For example, the user types:
  /aipic: a white cat sitting in a classroom drawing on the blackboard
  The system should:
  - Detect this command format (/aipic: prompt)
  - Extract the text prompt
  - Send it to an AI image generation API (e.g., DALL·E, Stable Diffusion,

DeepSeek)
    - Return the generated image (display in chat or save locally)

- **Topic 2: NLP Chat Keyword Extraction or Summary**
  Enable the system to analyze recent chat messages and extract keywords or generate a short summary.
  Example usage formats:
  /summary → returns a brief summary of recent chat
  /keywords → returns key topics or frequently mentioned words

  Note:
    - You may implement this using basic NLP tools (e.g., TF-IDF, spaCy, NLTK), or using API-based models like OpenAI / DeepSeek.
    - This feature should work on actual chat history in your system, not on fixed text.

- **Topic 3: Sentiment Analysis (Emotion Detection)**
  Enable the chat system to analyze the emotional tone of user's messages and display whether a message is Positive, Neutral, or Negative (or optionally more detailed emotions like Happy, Angry, Sad).
  When a user sends a message, the system should:
    - Analyze the sentiment of the message text
    - Detect the emotion type (e.g., 😊 Positive, 😐 Neutral, 😡 Negative)
    - Display the result in the chat interface (emoji, text tag, a special section in GUI or colored message bubble), and any form is acceptable as long as the sentiment result is clearly reflected in the system.

  Note:
    - You may use simple offline NLP tools like TextBlob, NLTK, or spaCy
    - Or use API-based models (e.g., OpenAI, DeepSeek, or HuggingFace)

4. Grading Policy

4.1 Compulsory Topic (50%) – Graphical User Interface

1. Baseline Functionality (40%)
The GUI must correctly send, receive, and display messages. It should update in real time and provide the essential features needed for a functioning chat client.

2. Innovative Features (10%)
Additional creative or advanced functionalities implemented on top of the baseline GUI.
Examples include:（You can implement one of them)

- Login and password system
- Emoji support
- File transfer
- Enhanced UI design

- Other thoughtful additions that improve usability or user experience

4.2 Selective Topic(s) (20%) + Bonus (up to 35%)

1. Completeness (10%): The selected topic is fully implemented. The feature behaves correctly and meets the expected goals.

2. Integration into the GUI (10%): The additional feature(s) must be properly integrated into your GUI. Users should be able to access and use the functionality through the interface.

3. Bonus Opportunities (up to 35%): You may earn up to 35% bonus by completing advanced features.

- Bonus points in Selective topics:
  - Chatbot Group Interaction (5%)
  - Interactive Multiplayer Gaming (5%)
- Complete any of the Bonus topics (10%): there are three bonus topics.

There are a total of **five Bonus Opportunities**, but **a maximum of four can be counted toward your grade**. Therefore, to receive the full **35% bonus**, it is recommended that you choose **the four options that best fit your project.**

4.3 Video Presentation (30%)

Your video presentation must be 10–15 minutes long and include the following four components:

1. Introduction (6%)

- Clearly introduce your project, including your goals and motivations.
- Provide context so your audience can understand the significance of your work.
- Be concise and direct.

2. Demo of the Application (6%)

- Showcase the usage of pi-mono as an AI code-assistant, e.g., how it generates/tests code (or documentation) for you.
- Showcase the major features of your GUI and selected topics.
- Demonstrate how the system works and what makes it effective or unique.
- Aim for an engaging, easy-to-follow demonstration.

3. Discussion (6%)

- Explain your development process:
  - Packages or libraries you used
  - Important design decisions
  - Significant changes made to the original chat system
- Help your audience understand the technical foundation of your work.

4. Analysis (6%)

- Reflect on your project:
  - Code organization
  - Class design and modular structure

    - Challenges you faced
    - Possible improvements or future expansions
- Show depth of thought and understanding.

5. Presentation Quality (6%)

Each section will also be assessed according to these criteria:

- Clarity: How easy it is to follow and understand your explanation
- Impressiveness: How effectively you demonstrate your technical skills
- Relevance: How closely your presentation aligns with your project goals

Note: Presentations shorter than 10 minutes or longer than 15 minutes may receive a deduction. Staying within the designated time frame and focusing on clarity, impressiveness, and relevance will help you deliver a strong and polished video.